HAKIMO Production Line App

Repo-based one-page summary generated from README, PROJECT_DOC, App.tsx, architecture docs, package.json, and Firebase setup files.

What It Is

An internal ERP-style web application for managing factory production operations, reporting, HR workflows, approvals, inventory, costs, and system administration. The repo shows a modular React 19 + TypeScript frontend backed by Firebase services and Zustand-based application orchestration.

Who It's For

Primary persona: a factory production supervisor or operations administrator who needs to monitor lines, capture daily production activity, and work inside a permission-controlled system. Adjacent personas in the repo include HR staff, system admins, quality users, inventory users, and repair teams.

What It Does

- Email/password authentication with Firebase Auth and route protection.
- Dynamic RBAC with default and custom roles, permission guards, and admin controls.
- Production operations across products, lines, plans, work orders, scanners, and daily reports.
- Role-based dashboards, KPIs, charts, printing, image/PDF export, and sharing workflows.
- HR workflows including attendance, leave, loans, payroll, approvals, and employee self-service.
- Inventory, costs, quality, repair, and system administration modules in the same app shell.
- Realtime updates, notifications, activity logging, and offline Firestore cache support.

How It Works

- **Frontend shell:** React 19 + Vite app with BrowserRouter in `App.tsx`; protected routes aggregate module route sets for dashboards, production, HR, costs, system, inventory, repair, online, and catalog areas.
- **Application layer:** Zustand store in `store/useAppStore.ts` orchestrates auth, permissions, subscriptions, notifications, and cross-module data loading.
- **Domain modules:** Feature code is organized under `modules/<domain>` for production, HR, inventory, quality, costs, repair, system, dashboards, and more.
- **Services/data access:** Repo architecture docs define the flow as UI -> store/use case -> services -> Firebase; service files wrap Firestore, Storage, and callable Functions access.
- **Platform services:** Firebase config in `modules/auth/services/firebase.ts` initializes Auth, Firestore with persistent local cache, Storage, and callable Functions (`us-central1`).
- **Event flow:** Shared event bus in `shared/events/event-bus.ts` lets modules emit and react to system events without blocking callers.

How To Run

- Install dependencies: `npm install`
- Create `.env.local` with the required `VITE_FIREBASE_*` values listed in `README.md`.
- Run the app: `npm run dev` (Vite dev server is configured for port 3000).
- Optional for deployment/runtime parity: build Functions with `npm --prefix functions run build`.

Not found in repo: explicit SLA, hosting URL for this exact environment, database schema diagram, or a dedicated end-user product positioning page.